Slide 3: Cache Optimization, Final Gather & Verification

Speaker C: Fuzail

⚡ i-k-j Compute Core

With data generated, isolated math begins immediately.

matmul.c — Cache Tiling

```
// Cache-friendly i-k-j loop order
for (int i = 0; i < my_rows; i++) {
    for (int k = 0; k < n; k++) {
        double a_val = local_a[i * n + k];
        for (int j = 0; j < n; j++) {
            local_c[i * n + j] +=
                a_val * mat_b[k * n + j];
        }
    }
}
```

🧠 **SPATIAL LOCALITY GUARANTEE**

By hoisting `a_val` and iterating over `j` internally, the hardware prefetcher reads `mat_b` linearly (stride-1). This drastically reduces L1/L2 cache misses compared to naive i-j-k loops.

📩 The Only Comm.

Network utilization remains strictly at **0%** until this exact line of code.

matmul.c — Final Gather

```
// Rank 0 allocates master buffer
double *mat_c = NULL;
if (rank == 0) mat_c = alloc_matrix(n,n);

MPI_Gatherv(local_c, my_rows * n, MPI_DOUBL
            mat_c, sendcounts, displs,
            MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

SENDING LOCAL_C SLICES

Rank 0	Rank 1	Rank 2	Rank 3
0..N/P	N/P..	2N/P..	..N-1

↓ Gatherv ↓

🗄 Rank 0 (Master) — mat_c

R0 | R1 | R2 | R3

✅ 100% Deterministic

Once Gathered, Rank 0 verifies mathematical integrity against the baseline.

matmul.c — fmod Verify

```
if (rank == 0) {
    double checksum = 0.0;
    for (int i = 0; i < n * n; i++) {
        checksum += mat_c[i];
    }

    // Constrain checksum bounds
    checksum = fmod(checksum,
                    18446744073709551616.0);

    printf("Checksum: %f\n", checksum);
}
```

- **fmod Constraining**
  By applying a 64-bit floating-point modulo (`fmod`), we ensure massive results do not overflow, allowing exact 1:1 validation.

✓ Matches Assigned Constraints perfectly